

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2333 — Sistemas Operativos y Redes — 2/2025

Tarea 0

Miércoles 20-Agosto-2025

Fecha de Entrega: Miércoles 03-Septiembre-2025 a las 22:00
Composición: Tarea en Parejas

Objetivos

- Utilizar *syscalls* para construir un programa que administre el ciclo de vida de un conjunto de procesos.
- Establecer un mecanismo de comunicación entre procesos mediante señales.

`DCControl`

A bordo de la Estación Espacial DCC, el equipo de control de misión ha detectado un problema: las misiones científicas que se ejecutan en paralelo están comenzando a saturar los sistemas de la estación. Para evitar un colapso del sistema, la capitana propone implementar **DCControl**: un programa de control de procesos que administre y supervise todas las misiones de la estación de forma simultánea. Tu tarea será desarrollar este sistema de control, capaz de lanzar nuevas misiones, monitorearlas y abortarlas en caso de sobrepasar el tiempo máximo asignado. Para lograrlo, deberás implementar un intérprete de comandos, también conocido como *shell*, que deberá cumplir con los siguientes requisitos:

- El programa principal **DCControl** no debe bloquearse. Se espera que los programas externos se ejecuten **en paralelo** mediante la creación de múltiples ~~misiones~~ procesos.
- Permitir el envío de señales a distintos procesos según sea necesario.
- Utilzar las *syscalls* vistas en clases.
- Garantizar que no queden procesos *zombie* o huérfanos en ningún caso de uso.

Funcionalidad del programa

Las principales funciones que debe cumplir el programa son:

1. El programa deberá manejar diversos comandos y ejecutar múltiples programas externos **de manera paralela** mediante la creación de procesos independientes. Además, deberá proporcionar el *feedback* correspondiente a cada comando cuando sea necesario.
2. Cuando un proceso creado alcance el tiempo límite `<time_max>`, que corresponde a un parámetro entregado al ejecutar el programa, se le enviará una señal `SIGTERM` para notificarle que tiene 5 segundos para finalizar su ejecución. En caso de no terminar después de este tiempo, se enviará la señal `SIGKILL` para forzar su terminación. Esta acción solo ocurrirá si hay procesos en ejecución. Es importante destacar que el programa principal continuará procesando comandos mientras esto sucede.
3. Al finalizar el programa principal, se tiene que mostrar en consola las estadísticas de los procesos ejecutados.

Comandos del programa

Los comandos que su *shell* debe soportar son:

- `launch <executable> <arg1> <arg2> ... <argn>`: Este comando toma la ruta *executable* de un ejecutable y los argumentos *argi* correspondientes a este ejecutable y lo ejecuta mediante un nuevo proceso, distinto al de la *shell*. En caso de que el ejecutable no exista, se le debe indicar el error al usuario. **Correr uno o varios programas no debe congelar su *shell***. Deberá investigar qué *syscalls* utilizar para cumplir con este requerimiento.

- `status`: Este comando entrega al usuario un listado de **todos los programas que hayan sido ejecutados** desde `DCControl`, incluyendo tanto los que siguen en ejecución como los que ya han finalizado. A continuación, se detalla la información mínima a mostrar.

  - PID del proceso.
  - Nombre del ejecutable.
  - Tiempo de ejecución del proceso en segundos. [1]
  - `exit code`: Código de salida del proceso si éste ha terminado, -1 en caso contrario.
  - `signal value`: Valor de la señal recibida por el proceso. Si no recibió ninguna, será -1. Por ejemplo, si se emite una señal `SIGINT`, este valor será 2.

- `abort <time>`: Este comando permite terminar, luego de `<time>` segundos, todos los procesos que se estén ejecutando de manera concurrente en el programa principal **en el momento en que se invoque el comando**, ignorando los procesos que se inicien posteriormente. Antes de ejecutarse, se debe validar si existen procesos en ejecución.

  - Si no hay procesos en ejecución, se debe informar al usuario con el siguiente mensaje:

    ```
    No hay procesos en ejecución. Abort no se puede ejecutar.
    ```

  - En caso de que sí haya procesos en ejecución, se esperará hasta que transcurra el tiempo definido en `<time>`. Si un proceso no finaliza dentro de ese tiempo, se debe imprimir la siguiente información de los procesos a terminar, para después enviarles la señal `SIGTERM`:

    ```
    Abort cumplido.
    PID nombre_del_ejecutable tiempo_ejecución exit_code signal_value
    ```

- `shutdown`: Este comando permite terminar el programa principal `DCControl`. Al invocar el comando, se debe verificar si hay procesos en ejecución.

  - Si hay procesos en ejecución al momento de invocar el comando, se envía la señal `SIGINT` a todos los procesos que estén ejecutándose en ese momento. El programa no debe bloquearse y debe continuar aceptando comandos durante los 10 segundos siguientes, tiempo durante el cual pueden crearse nuevos procesos que no recibirán esa señal inicial. Si al terminar ese periodo algún proceso sigue activo, se enviará `SIGKILL` para forzar su terminación. Luego, se imprimirán las estadísticas de **todos los procesos** y el programa finalizará. En este escenario, cualquier comando `abort`, ya sea previamente programado o ejecutado durante el tiempo de espera del `shutdown`, queda anulado.
  - Si no hay procesos en ejecución al momento de invocar el comando, el programa imprimirá las estadísticas de **todos los procesos** y terminará inmediatamente.

[1] Para obtener el tiempo en segundos pueden utilizar time

Las estadísticas a imprimir en ambos casos son las siguientes:

```
DCControl finalizado.
PID nombre_del_ejecutable tiempo_ejecución exit_code signal_value
```

- `emergency`: Este comando termina inmediatamente todos los procesos que estén ejecutándose en el momento de su invocación, enviándoles la señal `SIGKILL` de forma directa. No se otorgará tiempo de gracia ni se enviarán señales previas. Luego, se imprimen las estadísticas de **todos los procesos**, y el programa principal `DCControl` finaliza su ejecución:

```
¡Emergencia!
DCControl finalizado.
PID nombre_del_ejecutable tiempo_ejecución exit_code signal_value
```

Supuestos

Se pueden realizar los siguientes supuestos:

- Siempre se entregará un comando válido.
- Siempre se entregarán la cantidad de argumentos requeridos.
- No es necesario considerar el caso en que `<time_max>` y `abort <time>` finalicen exactamente al mismo tiempo.
- Por cada ejecución de `DCControl`, podrán haber como máximo 10 procesos en ejecución de manera simultánea.
- Por cada ejecución de `DCControl`, se ejecutará solo una vez el comando `shutdown`.

Ejecución

El programa principal será ejecutado por línea de comandos con la siguiente sintaxis:

```
./DCControl <time_max>
```

Donde `<time_max>` es un parámetro entero **opcional**. Este indica la cantidad máxima de segundos que puede demorarse en correr un proceso antes de que sea terminado. Si no se entrega este parámetro, se debe considerar que los procesos tienen tiempo ilimitado para ejecutar.

Al iniciar, `DCControl` quedará a la espera de comandos ingresados por el usuario. Algunos ejemplos de comandos válidos son:

```
launch sleep 20
launch ls
status
shutdown
```

Formalidades

A cada alumno se le asignó un nombre de usuario y una contraseña para el servidor del curso[2]. Para entregar su tarea usted deberá crear una carpeta llamada `T0` en el directorio principal de su carpeta personal y subir su tarea a esa carpeta. En su carpeta `T0` **solo debe incluir el código fuente** necesario para compilar su tarea, además del reporte y un `Makefile`. Se revisará el contenido de dicha carpeta el día Miércoles 03-Septiembre-2025 a las 22:00.

- La tarea debe ser realizada solamente en parejas.
- La tarea deberá ser realizada en el lenguaje de programación **C**. Cualquier tarea escrita en otro lenguaje de programación no será revisada.
- **NO debe incluir archivos binarios**[3]. En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe incluir un repositorio de git**[4]. En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe usar VSCode para entrar al servidor**[5]. En caso contrario, tendrá un descuento de 0,2 puntos en su nota final.
- Si inscribe su grupo de forma incorrecta o no lo inscribe, tendrá un descuento de 0.3 décimas
- Su tarea debe compilarse utilizando el comando `make`, y generar un ejecutable llamado `DCControl` en esa misma carpeta. Si su programa **no tiene** un `Makefile`, tendrá un descuento de 0.5 décimas en su nota final y corre riesgo que su tarea no sea corregida.
- En caso de no cumplir con el formato de entrega, tendrá un descuento de 0.3 décimas en la nota final.
- Si ésta **no compila** o **no funciona** (*segmentation fault*), obtendrán la nota mínima, pudiendo recorregir modificando líneas de código con un descuento de una décima por cada cuatro líneas modificada, con un máximo de 20 lineas a modificar.

El no respeto de las formalidades o un código extremadamente desordenado podría originar descuentos adicionales, los cuales quedarán a discreción del ayudante corrector. Se recomienda modularizar, utilizar funciones y ocupar nombres de variables explicativos. En el caso de no entregar en la carpeta especificada la tarea, **no** se corregirá.

Evaluación

- **1.5 pts.** Correcta implementación de `launch` con múltiples procesos paralelos.
- **0.7 pts.** Correcta implementación de `<time_max>`.
- **1.0 pts.** Correcta implementación de `status`.
- **1.3 pts.** Correcta implementación de `abort`.
- **0.5 pts.** Correcta implementación de `shutdown`.
- **0.5 pts.** Correcta implementación de `emergency`.
- **0.5 pts.** Manejo de memoria. Se otorgará este puntaje si `valgrind` reporta 0 *leaks* y 0 errores de memoria en **todos los casos de uso**[6].

---

2 `iic2333.ing.puc.cl`
3 Los archivos que resulten del proceso de compilar, como lo son los ejecutables y los *object files*
4 Si es que lo hace, puede eliminar la carpeta oculta `.git` antes de la fecha de entrega.
5 Si es que lo hace, puede eliminar la carpeta oculta `.vscode-server` antes de la fecha de entrega.
6 Es decir, el código debe mostrar 0 *leaks* y 0 errores en todas las pruebas.

Política de atraso

Se puede hacer entrega de la tarea con un máximo de 2 días hábiles de atraso. La fórmula a seguir es la siguiente:

$$N_{T_1}^{\text{Atraso}} = \text{mín}(N_{T_1}, 7{,}0 - 0{,}75 \cdot d)$$

Siendo d la cantidad de días de atraso. Notar que esto equivale a un descuento *soft*, es decir, cambia la nota máxima alcanzable y no se realiza un descuento directo sobre la nota obtenida. El uso de días de atraso no implica días extras para alguna tarea futura, por lo que deben usarse bajo su propio riesgo.

Importante: Se recomienda incluir un archivo **README.md** en el que se indiquen las funcionalidades implementadas y/o cualquier consideración que deba tenerse en cuenta al momento de corregir la tarea. Además, en caso de no cumplir con el formato de entrega, tendrá un descuento de 0.3 décimas en la nota final.

Preguntas

Cualquier duda preguntar a través del foro oficial.